

ARTIFICIAL INTELLIGENCE

CMPU4010

Student Number: C21403052



TU Dublin
TU857/4

Part 1: Maze Path Finding with 3 Different Algorithms

1. Creating the Maze

To be able to test any path finding algorithm, the first task I took upon implementing was to create the maze at the top of the file. This included setting the maze size to 5 by 5. Then inputting the walls for which figure 1 I used as an example was squares (2,2), (2,3), (2,4), (3,4), (4,4), (4,3). These wall squares will act as the barriers for where the algorithms can't go.

GOAL				
			START	

Figure 1: Example Grid Layout

During the first implementation of these walls, I noticed I entered the wrong tile numbers which was later resolved in the testing of the move function.

```
maze_size(5,5) .  
walls(2,2) .  
walls(2,3) .  
walls(2,4) .  
walls(3,4) .  
walls(4,4) .  
walls(4,3) .
```

Figure 2: Maze Size and Wall Placements

2. Movement Method

For the movement of the assignment, I had to create a move function that provides four-directional movement (Left, Right, Up, and Down). The position is only able to move right if the addition of X is equal to or less than the width of the maze size and if a wall is not present in the next square. For left it's the same except instead of checking the maze size it just checks that the X minus one isn't less than one. For Up and down it does the same as right and left but for the height of the maze size instead of the width.

3. The Three Search Algorithms

3.1. Depth-First Search (DFS)

Through research of the search algorithm depth-first, it appears to use a systematic strategy for exploring the maze or graph by always visiting an unexplored square of the current node before back tracking. In simple terms it goes one way until it reaches an end then goes, back one and will continue another way and repeat this process until it either explores all the maze or reaches a point where it gets stuck. This algorithm

seems to present a few problems and does not seem to be optimal when searching for the shortest path. It can also end up in an infinite loop in certain graphs.

For the implementation of this search algorithm, I started with:

Defining the predicate `path_dfs` which takes the starting position, goal position and will output the path. It calls the helper `dfs/4`, which takes the sets the starting, goal positions, and an empty list for the visited squares, and a variable `Rev` to store the reversed path.

Then creating the `dfs/4` helper. To make the helper, I first needed to define the base-case which is just `dfs(State, State, Vis, [State|Vis])`. This just basically defines if the current state is the same as the goal state then prepend it to the visited and return back to the `path_dfs` predicate.

Then creating the recursive clause for if the start isn't the goal, the `move/2` function is called which enumerates one neighbour using the `Next` variable so if left go left, if not right, if not up and if not down in that order, because of the four move clauses in `move/2`. If the square is available but is also in the visited list then skip and continue. It will recurse with `Next` as the new `State` and push the current square to the visited list. Then when the base-case is successful, it will unwind through the recursive calls to (1), where `reverse/2` flips the backward list into start to goal order.

3.2. Iterative-Deepening Search (IDS)

The IDS algorithm seems to perform depth-limited search, starting with a base depth of zero and increasing it one level at a time until it discovers the goal. It combines the DFS and Breadth-First Search algorithms to guarantee finding the path from the start to goal while also providing an optimal path being the shortest.

For implementation of this code into the assignment I first declared the entry point being the `path_ids/3`. This is what will be called to start the first depth limit at 0 and delegates the iterative depths `id/4`. This predicate takes the Start and Goal positions and will output the Path.

Following this I proceeded to create the outer iterative loop `id/4` which will try the current depth limit and call the `dls/5` function with the current Limit, an empty list and will then return the results in `Rev` variable. If the `dls/5` reaches the goal inside the limit provided, `reverse/2` flips the reversed path forward and finishes outputting the path.

However, if the goal is not reached and no path is found at the current depth, there needs to be a second clause of `id/4`. This clause created will increase the limit and recursively call itself.

Here I created the Base case for the depth limited search called the `dls/5`. The base case defines if the current state is the goal then it successful.

Then if that fails the recursive case starts until the base case is successful. To create the recursive case, I have to take in the current state or position of the square, the

position of the goal, the current limit, and the visited squares list. Then check that the limit is greater than 0, then decrement the limit by 1. Generate a neighbour using the move function. Skip if the neighbour has already been visited by checking the visited list, and then recurse with the Next variable holding the next position and the shortened limit until the goal is found by the base case succeeding or if L hits zero, the guard fails and back tracking to try the next neighbour not already visited until there are no more neighbours, which will propagate back up to the id/4 case.

3.3. A* Chebyshev Heuristic

Now initially I attempted implementing the A* Manhattan Heuristic Search within this assignment but failed on multiple attempts and a few hours of logical steps on overcoming an error that prevented me from running the path. I have left the code within assignment prolog file as path_a. I believe the error did stem from the compare_f function being poorly implemented as I took reference from a GeekstoGeeks form discussion about the algorithm. Here is a screenshot of the error encountered if you're curious.

```

?- path_a(pos(3,3),pos(1,5),Path).
ERROR: No rule matches lists:min_member(user:compare_f,[[pos(3,3),0,4,[pos(3,3)]]],_1970)
ERROR: In:
ERROR: [14] lists:min_member(user:compare_f,[[...]],_2072)
ERROR: [13] a([[...]],pos(1,5),_2118) at c:/users/aaronburton/desktop/college work/year 4 semester 2/artificial intelligence/assignment.pl:80
ERROR: [12] path_a(pos(3,3),pos(1,5),_2184) at c:/users/aaronburton/desktop/college work/year 4 semester 2/artificial intelligence/assignment.pl:75
ERROR: [11] toplevel_call(user:...) at c:/program files/swipl/boot/toplevel.pl:1317
Exception: (4) setup_call_cleanup('$_toplevel':notrace(call_repl_loop_hook(begin, 0)), '$_toplevel':$_query_loop'(0), '$_toplevel':notrace(call_repl_loop_hook(end, 0))) ? creep
?-

```

Figure 3: A* Manhattan Comparator Error

But besides my failed attempt at implementing the Manhattan Heuristic, I proceeded to attempt another A* Chebyshev Heuristic. From research of the Chebyshev Heuristic, it is still admissible and consistent for a 4-way grid. Although it is slightly less informed than the Manhattan approach, it avoids the DWIM confusion experienced trying to implement Manhattan and keeps the A* optimal.

Replacing the priority-queue step with a keysort/2 approach eliminates the need for a comparator case. Making the implementation of it after attempting the Manhattan simpler.

Implementation of this began with developing the predicate path_a2. This would take the Starting and goal coordinates and return the most optimal path identified. It calls the case h_cheb which calculates the Chebyshev distance between the start and goal as the initial estimate H0. Then calls the next predicate astar2 which takes the initial heuristic estimate, starting position, the initial cost from start to start being 0 and the initial path [Start] containing the start state. The astar2 predicate will perform the A* search. Then after the search the variable Rev holding the reversed path A* found is reversed, using the reverse function.

Then continuing on to create the astar2 base case where if the head of the Open list contains the Goal, the base case is successful, and the path is extracted from the Open list. If not it continues to the recursive case.

The recursive case was similar to the Manhattan recursive case made except instead of the comparator case, a keysort was implemented. This sorted the Open list based on

the first element in the key, which is the heuristic. The sorted then gets the node with the lowest F-cost from the sorted Open list.

Then I extracted the state, and the accumulated path from the best node found.

This part took some time figuring out is how the findall is used and implemented correctly to find all the neighbouring states of the current state. First using the move to find the Next of the current state. Then check the neighbouring state Next is not already accumulated in the path Acc to avoid potential loops. Which I learned the hard way. Then increment the G by one being G1

Then using the h_cheb case to calculate the heuristic estimate from the neighbor Next to the Goal. Then calculate the F-cost for the neighbour Next with F1 being the G1 plus the heuristic estimate. Then I used the findall to collect all the children's nodes into the Children list where each child is representing as F1-[Next, G1, [Next|Acc]].

Then I had to combine the remaining nodes into Rest from the Open list with the newly generated children list to create the new Open1 list with the append function.

Then to finish the case I called itself to recursively call astar2 with the updated Open1 list to continue the search until it either fails or the base case is successful.

Part 2 Varying Terrain

1. Addition Of Water Squares

First thing to introduce the implementation of water terrain and energy into the prolog file is to create a new predicate water/2 with the location of what squares are affected. I chose the squares (3,2) and (4,2) on the grind to be converted into water squares. Now in the assignment to cross these squares I have to implement a movement and cost model.

2. Movement

To implement this new energy aware search, I need to make it so that any movement now costs one energy and movement over a water square cost three. So, instead of editing the move/2 predicate, I decided to define a wrapper predicate step/3. The step/3 predicate takes the current position and the second position and calculates the energy for stepping onto the next position. It wraps the move/2 predicate and checks if it's a water square then defines C the cost as three, and if not then as one.

```
%Calculate each steps energy
step(pos(X,Y), pos(X1,Y1), C) :-
    move(pos(X,Y), pos(X1,Y1)),
    (   water(X1,Y1) -> C = 3 ; C = 1 ).
```

Figure 4: Step wrapper predicate

3. Algorithm Updates for Energy

3.1. Calculating the energy for each path

I proceed to create a predicate to calculate the total energy cost of a given path, the predicate takes the path and outputs the energy cost. The base case of this predicate is if the path has only one node has an energy cost of 0.

The recursive case for the energy cost of a path takes the first two nodes as A and B and then stores the rest of the nodes in the list Rest. It then calls the predicate step to get the cost between the two squares and assigns it to C. Then recursively calculates the energy E1 of the rest of the path from B onwards, so B and C would become A and B. It then at the end calculates the total energy E by adding the cost C of the step from A to B to energy E1 of the rest of the path.

3.2. Show the energy for each path function

To show the energy outputs for the search algorithm predicates DFS and IDS, I created two predicates called path_dfs_e and path_ids_e. These both take in the start and goal coordinates and output the path and energy cost for those paths. Then call the predicates path_dfs and path_ids with the path_energy I created earlier.

```
%Show the energy for each path function
path_dfs_e(Start, Goal, Path, Energy) :- path_dfs(Start, Goal, Path), path_energy(Path, Energy).
path_ids_e(Start, Goal, Path, Energy) :- path_ids(Start, Goal, Path), path_energy(Path, Energy).
```

Figure 5: DFS and IDS energy path predicates

3.3. A* Chebyshev Heuristic Alterations

To be able to calculate and output the energy for each path with the A* Chebyshev Heuristic predicate require some alterations to the code. First of all, I needed to add Energy into the path_a2 predicate and then update the astar2 to also include the energy input and outputs. Then update the move predicate with the step predicate to calculate the cost of energy for each move. Then when calculating the G1 cost instead of incrementing it by one, it now increments it by the cost calculated through the step predicate. Then finally update the astar2 output at the recursive call to output the Energy as well.

3.4. Lowest Energy Cost Search

The heuristic calculates the shortest path regarding steps but not of cost of energy. For this I decided to create a new predicate called path_lec/2. This predicate takes in the coordinates for the starting position and goal position and outputs the path with smallest amount of energy. It calls the base predicate lec/3 with an initial Open list containing the starting node and its initial cost 0. It then reverses the path obtained from the lec predicate and returns the reversed path as path.

To continue I then defined the predicate `lec/3` base case to see if the first element in the Open list contains the Goal position, the search is successful, and the accumulated path (Acc) and its cost of energy (Energy) are outputted.

The recursive predicate takes the Open list nodes or squares, the goal position, and will output the path and the energy cost. Similar to how the A* Chebyshev Heuristic was made, it uses the `keysort` to sort the Open list of nodes based on its energy cost, then takes the state, accumulated cost and accumulated path from the Best node. Then using the `findall` to find the neighbouring squares of the current position/ state and calls the `step/3` predicate to get the cost of moving to the next step and moving the position to the next square, while also checking that the next step isn't already visited by checking the (Acc) accumulated path. It then calculates the accumulated cost (G1) to reach the neighbour square. And then sets the energy cost (Energy1) to be the accumulated cost G1, to minimize the path cost. After the `findall` function collects all the children's nodes into the Kids list, the recursive case then combines the remaining nodes in the Rest list from the Open list with the newly generated Kids list to create the updated Open1 list, using the `append` prolog function. This recursive case will recursively call the `lec` predicate with the updated Open1 list continue the search for the path with the smallest energy cost until the base case is successful.

```
% _____ Lowest Energy Cost Search _____
path_lect(Start, Goal, Path, Energy) :-
    lec([0-[Start,0,[Start]]], Goal, Rev, Energy),
    reverse(Rev, Path).

lec([Energy-[Goal,_,Acc]|_], Goal, Acc, Energy).
lec(Open, Goal, Path, Energy) :-
    keysort(Open, [ _-Best | Rest ]),
    Best = [State, G, Acc],
    findall(Energy1-[Next, G1, [Next|Acc]],
        (
            step(State,Next,C),
            \+ member(Next, Acc),
            G1 is G + C,
            Energy1 = G1),
        Kids),
    append(Rest, Kids, Open1),
    lec(Open1, Goal, Path, Energy).
```

Figure 6: Lowest Energy Cost Predicates

4. Testing Logs

1. Initial test was the evaluating the move predicate, to do this I put I called `move(pos(4,1), Next)`. This should output 3 squares (5,1), then (3,1), and then (4,2). The output was:

```
?- move(pos(4,1), Next).  
Next = pos(5, 1) ;  
Next = pos(3, 1) ;  
Next = pos(4, 2) ;  
false.
```

Figure 7: Testing Movement

This test was successful.

2. Again, testing the movement, I tested that when the position is set to (3,3) it should only output (3,2), the results were:

```
?- move(pos(3,3), Next).  
Next = pos(3, 4) ;  
Next = pos(3, 2).
```

Figure 8: Movement Test Error

The test was unsuccessful and Identified that I entered the wrong values for the walls. This has now been resolved, and the new output is:

```
?- move(pos(3,3), Next).  
Next = pos(3, 2).
```

Figure 9: Movement Testing Corrected Output

3. The next test conducted was on the `path_dfs` predicate, for this we'll start in the middle of the grid (3,3) and have the goal set to (1,5). The path was ran using the command: `path_dfs(pos(3,3), pos(1,5), Path)`. The results were:

```
?- path_dfs(pos(3,3), pos(1,5), Path).  
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;  
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,1), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;  
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(5,1), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;  
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;  
Path = [pos(3,3), pos(3,2), pos(3,1), pos(4,1), pos(5,1), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;  
Path = [pos(3,3), pos(3,2), pos(3,1), pos(4,1), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;  
Path = [pos(3,3), pos(3,2), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;  
false.
```

Figure 10: Dfs path results

Here we can see the search algorithm branch out on multiple paths until the end where it finds a path that is successful in reaching the goal.

4. The following test was conducted to check that the IDS search algorithm could obtain the optimal path for the same coordinates and output the other paths as well. The

command used in this test was: `path_ids(pos(3,3), pos(1,5), Path)`. The results were:

```
?- path_ids(pos(3,3), pos(1,5), Path).
Path = [pos(3,3), pos(3,2), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,1), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(4,1), pos(5,1), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(4,1), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,1), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(4,1), pos(5,1), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(4,1), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,1), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
```

Figure 11: IDS results

The ids results were successful in determining the shortest path, although there was a duplicate output of the same optimal path, it then proceeded get the second shortest path and then third, and continued on until no more paths were available so it started repeating itself before being forced to end.

- While trying to attempt to get the A* Manhattan Search working through test, and comparator mismatch kept causing errors, from changing it from a one line argument to three and back it repeatedly outputted the same error as show below by the results:

```
?- path_a(pos(3,3), pos(1,5), Path).
ERROR: No rule matches lists: min_member(user:compare_f, [[pos(3,3), 0, 4, [pos(3,3)]], _1970)
ERROR: In:
ERROR: [14] lists: min_member(user:compare_f, [[...], _2072)
ERROR: [13] at([...], _2119) at c:/users/aaroburton/desktop/college work/year 4 semester 2/artificial intelligence/assignment.pl:80
ERROR: [12] path_a(pos(3,3), pos(1,5), _2164) at c:/users/aaroburton/desktop/college work/year 4 semester 2/artificial intelligence/assignment.pl:75
ERROR: [11] top_level_call(user:..., ...) at c:/program files/swipl/boot/toplevel.pl:1317
?- Exception: (4) setup_call_cleanup('stoplevel' notrace(call_repl_loop_hook(begin, 0)), 'stoplevel': 'query_loop'(0), 'stoplevel' notrace(call_repl_loop_hook(end, 0))) ? creep
```

Figure 12: A* Manhattan Search Comparator Error

- Continuing on to test the A* Chebyshev Heuristic Search, the `path_a2(pos(3,3), pos(1,5), Path)`. Was inputted and the results were:

```
?- path_a2(pos(3,3), pos(1,5), Path).
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(4,1), pos(4,2), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(5,2), pos(5,1), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(4,2), pos(4,1), pos(3,1), pos(2,1), pos(1,1), pos(1,2), pos(1,3), pos(1,4), pos(1,5)] ;
Path = [pos(3,3), pos(3,2), pos(3,1), pos(4,1), pos(5,1), pos(5,2), pos(5,3), pos(5,4), pos(5,5), pos(4,5), pos(3,5), pos(2,5), pos(1,5)] ;
false.
```

Figure 13: A* Chebyshev Heuristic Test

Here we can see the two shortest paths that are available from the starting position to the goal position and then the shortest paths found presented after until no more paths can be found, and it returns false.

- Now onto the energy testing, first we'll test that the step outputs different energy cost depending on if the square is regular square or water square. To test this, I inputted the command: `step(pos(3,3), pos(3,2), Cost)`. Which steps into a water square and `step(pos(1,5), pos(1,4), Cost)`. Which steps into a regular square, the results were:

```
?- step(pos(3,3), pos(3,2), Cost).
Cost = 3.

?- step(pos(1,5), pos(1,4), Cost).
Cost = 1.
```

Figure 14: Step testing results

Here we can see that the step predicate is working as intended.

8. Next the testing of the path energy functions to ensure that they output the energy along with the path, I first tested the path_dfs_e predicate with the points (3,3), and (1,5), and Path and Energy. The results were as follows:

```
?- path_dfs_e(pos(3,3), pos(1,5), Path, Energy).
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,1),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(5,1),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(4,1),pos(5,1),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(4,1),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 10 ;
false.
?- 
```

Figure 15:Energy test on path dfs

The results output the path and energy and stop when the shortest path is found.

9. Continuing to the next predicate to test which is the path_ids_e predicate with the same inputs as the dfs energy predicate. The results were as follows:

```
?- path_ids_e(pos(3,3), pos(1,5), Path, Energy).
Path = [pos(3,3),pos(3,2),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 10 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 10 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 10 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 10 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,1),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(5,1),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(4,1),pos(5,1),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(4,1),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 16 ;
false.
```

Figure 16: Energy Test on the IDS path

The results here show the ids outputting the energy with the path as intended making the test successful.

10. Now onto the testing of the updated A* Chebyshev search algorithm to include the energy output, the same parameters were inputted into the path_a2 as before and the results were as follows:

```
?- path_a2(pos(3,3), pos(1,5), Path, Energy).
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(4,1),pos(5,1),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(4,1),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,1),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)],
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(5,1),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)],
Energy = 16 ;
false.
```

Figure 17: A* Chebyshev Energy Output Test

Here we can see the results have the energy output we are looking for and are still focusing on the shortest path but not the path that cost the least amount of energy explaining why the shortest energy path was not show.

11. Finally, the test of the lowest cost search algorithm implemented based off the A* heuristic, this will test that the output will find the optimal path for the least amount of energy. The command used is the `path_lec(pos(3,3), pos(1,5), Path, Energy)`. And the results were as follows:

```
?- path_lec(pos(3,3), pos(1,5), Path, Energy).
Path = [pos(3,3),pos(3,2),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)].
Energy = 10 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)].
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)].
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(4,1),pos(5,1),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)].
Energy = 14 ;
Path = [pos(3,3),pos(3,2),pos(3,1),pos(4,1),pos(4,2),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)].
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(5,2),pos(5,1),pos(4,1),pos(3,1),pos(2,1),pos(1,1),pos(1,2),pos(1,3),pos(1,4),pos(1,5)].
Energy = 16 ;
Path = [pos(3,3),pos(3,2),pos(4,2),pos(4,1),pos(5,1),pos(5,2),pos(5,3),pos(5,4),pos(5,5),pos(4,5),pos(3,5),pos(2,5),pos(1,5)].
Energy = 16 ;
false.
```

Figure 18: LEC Output Test

Here we can see the results outputting the path with the least cost for energy and paths with the second and third and so one least cost of energy following.

12. Final Test for the changing of the layout of the map will all the paths still work. I have updated the map to look like the figure below with water at (1,2), and (8,10).

								WATER		
								GOAL		
WATER										
START										

Figure 19: Second Map

The results were as follows:

a. Path_dfs_e:

[illegible]

b. Path_ids_e:

```
7- path_ids = pos(1,1).pos(8,6). Path_Energy:
Path = [pos(1,1).pos(1,2).pos(1,3).pos(1,4).pos(1,5).pos(2,5).pos(3,5).pos(4,5).pos(5,5).pos(5,6).pos(5,7).pos(5,8).pos(5,9).pos(5,10).pos(6,10).pos(7,10).pos(8,10).pos(9,10).pos(9,9).pos(9,8).pos(9,7).pos(9,6).pos(8,6)].
Energy = 26 ;
```

c. Path_a2 and path_lec:

With the size of this maze it take a far longer time to run both of these paths due to the keysort heuristic, it would need far greater processing power due to the thousands of possibilities it will need to calculate.